Multi-Tenant Mobile Antenna Placement Optimization

PLR — Programação em Lógica com Restrições · FEUP 2026

Martim Pinheiro (up20509641) Adam Oprchal (up202512712)

Faculdade de Engenharia da Universidade do Porto

June 2026

Outline

Problem Overview

Multi-Tenant Mobile Antenna Placement Optimization

Given a **grid map** with varied terrain, we must:

- Place **cell towers** on valid terrain cells
- Assign **antennas** from 4 telecom providers to towers
- **Minimise total infrastructure cost** while maximising signal coverage

Key Characteristics

- Towers can be **shared** by up to 4 tenants (multi-tenant)
- Each provider has a different **frequency band** and **coverage radius**
- Uncovered cells incur a **financial penalty** (soft constraint)

Terrain Types

Terrain	Tower Cost	Symbol
Plains	€1 000	`.`
Forest	€1 500	`F`
Building	€2 500	`B`
Mountain	*Banned*	`M`

Table 1: Terrain costs

Additional Costs

- **Antenna slot**: €100 per antenna
- **Uncovered cell**: €800 per provider

Penalty makes coverage a **soft constraint**—the solver can choose not to cover a remote cell if it is cheaper than building a tower for it.

Provider Frequency Bands

Provider	Band	Manhattan Radius	Characteristics
P0	High-band / 5G mmWave	0	Own cell only; high capacity
P1	Mid-band / 4G–5G	1	Adjacent cells; balanced
P2	Mid-band / 4G–5G	1	Adjacent cells; balanced
P3	Low-band / Macro LTE	2	Wide area; legacy coverage

Table 2: Provider coverage parameters

Note

P1 and P2 share the same radius, introducing **solution symmetry**—any swap of their placements yields an equally valid solution.

Outline

Decision Variables

Let $N = R \times C$ be the total number of grid cells.

$t_i \in \{0, 1\}$	tower built at cell i, $\quad i \in [0, N)$
$a_{k,i} \in \{0, 1\}$	provider k has antenna at cell i
$s_{k,j} \in \{0, 1\}$	slack: cell j uncovered by provider k

where $k \in \{0, 1, 2, 3\}$ and j ranges over all non-banned cells.

Coverage Sources

$\text{cov}(k, j) = \{i \in [0, N) \mid \|i - j\|_1 \leq r_k \wedge i \notin \text{banned}\}$

Constraints

C1 — No tower on mountain:

$$t_i = 0 \qquad \forall\, i \in \text{banned}$$

C2 — Antenna requires a tower:

$$a_{k,i} \leq t_i \qquad \forall\, k, \forall\, i$$

C3 — Multi-tenant capacity (max 4 providers per tower):

$$\sum_{k=0}^{3} a_{k,i} \leq 4 \qquad \forall\, i$$

C4 — Coverage or penalty (reified):

$$\sum_{i \in \text{cov}(k,j)} a_{k,i} + s_{k,j} \geq 1 \qquad \forall\, k, \forall\, j \notin \text{banned}$$

Objective Function

Minimise total infrastructure cost:

$$\min \underbrace{\sum_{i} c_i \cdot t_i}_{\text{tower construction}} \underbrace{+ 100 \cdot \sum_{k,i} a_{k,i}}_{\text{antenna slots}} + \underbrace{800 \cdot \sum_{k,j} s_{k,j}}_{\text{uncoverage penalties}}$$

where $c_i \in \{1000, 1500, 2500\}$ depends on terrain type.

Problem Class

This is a **Constraint Satisfaction & Optimisation Problem (CSOP)** over Boolean finite domains—naturally suited to Constraint Programming.

Outline

OR-Tools CP-SAT

Platform: Google OR-Tools CP-SAT · Python 3

Model construction:

- `NewBoolVar` for every t_i, $a_{k,i}$, $s_{k,j}$
- Coverage sources precomputed before model building
- Linear constraints added with `model.Add(...)`
- Minimisation via `model.Minimize(...)`

Key snippet

```
for (k,tgt), srcs in coverage_sources:
  model.Add(
    sum(ant[k][s]
     for s in srcs)
    + slack[k][tgt] >= 1)
```

Solver configuration

`CpSolver` with `max_time_in_seconds` timeout; accepts first OPTIMAL or FEASIBLE status.

SICStus Prolog CLP(FD)

Platform: SICStus Prolog 4.x · `library(clpfd)`

Declarative model:

- Lists of 0/1 domain variables: `T`, `P0–P3`, `S0–S3`
- `scalar_product` for cost terms
- `sum(..., #>=, 1)` for coverage
- Branch-and-bound via `labeling([minimize(GlobalCost)], Vars)`

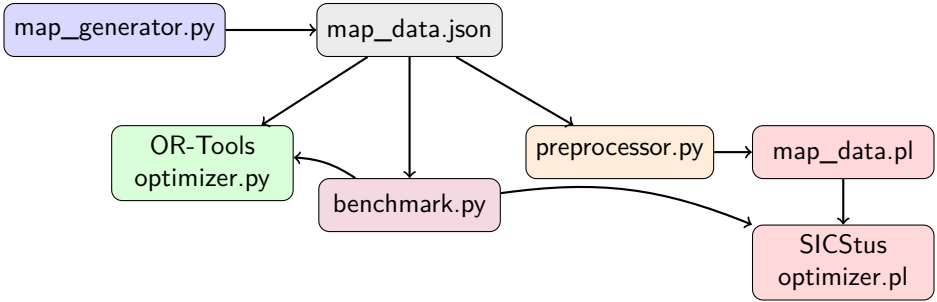
Key snippet

```
sum([Slack|CovVars],
  #>=, 1)
labeling(
  [minimize(Cost)],
  AllVars)
```

Preprocessor

A Python script (`preprocessor.py`) translates `map_data.json` into Prolog facts (`map_data.pl`), keeping a single source of truth for map data across both solvers.

Architecture



Outline

Benchmark Setup

Test matrices:

- Grid sizes: 4×4 to 8×8
- 3 random seeds per size (42, 123, 456)
- Timeout: **60 seconds** per run
- 15 total runs per solver

Metrics collected:

- Wall-clock time (ms)
- Optimal cost (€)
- Coverage (%)
- Solution equivalence check

Correctness validation

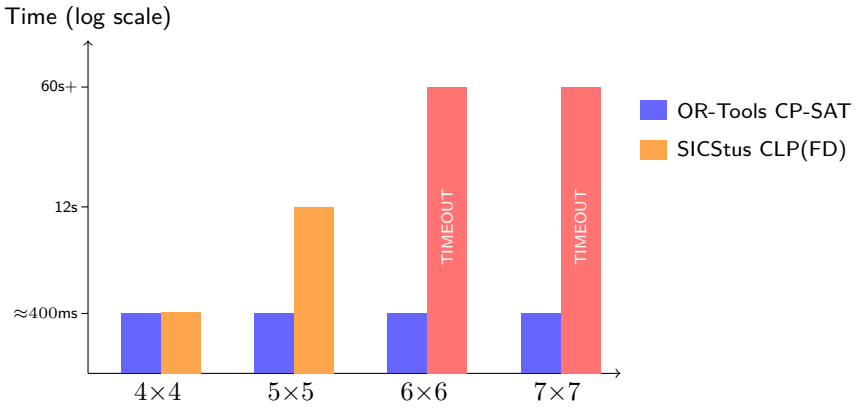
For instances where both solvers complete, their optimal costs are compared. All 6 completed instances (three 4×4, three 5×5) produce **identical costs**, confirming model equivalence.

Benchmark Results

Grid	Seed	OR-Tools		SICStus	
		Time (ms)	Cost (€)	Time (ms)	Cost (€)
4×4	42	338	13 600	371	13 600
4×4	123	342	14 000	383	14 000
4×4	456	344	14 000	397	14 000
5×5	42	332	22 500	13 850	22 500
5×5	123	337	21 500	8 873	21 500
5×5	456	327	21 500	13 329	21 500
6×6	42	373	31 600	TIMEOUT	—
6×6	123	349	31 600	TIMEOUT	—
6×6	456	347	31 200	TIMEOUT	—
7×7	all	≈390	≈42 700	TIMEOUT	—
8×8	all	≈440	≈55 700	TIMEOUT	—

Table 3: Benchmark summary (60 s timeout)

Solve Time Comparison



Solution Quality

Coverage

Both solvers achieve approximately **82–84%** coverage across all tested instances.
The remaining **16–18%** of cells are intentionally left uncovered—it is cheaper to pay the €800 penalty than to build a tower tha would only serve remote cells.

Model Equivalence

On all 6 instances where both solvers terminate:

- **Identical optimal costs**
- **Same coverage %**

This confirms that both implementations encode **exactly the same CSP**, despite using different constraint programming paradigms.

Outline

Why is SICStus Slower?

Four compounding factors explain the performance gap:

1. **Naive variable ordering** in `labeling/2`
2. **Weak lower bounds** in branch-and-bound
3. **CLP(FD) arc consistency vs CDCL** in CP-SAT
4. **Unhandled symmetry** between P1 and P2

Root cause

These are not fundamental limitations of CLP(FD)—they are **configuration choices** in this solver setup, each fixable with targeted improvements.

Factor 1 — Naive Variable Ordering

Current labeling:

- All tower vars first, then all antenna vars, then all slacks
- Solver commits to a full tower layout *before* considering coverage
- Conflicts discovered deep in the tree ⇒ expensive backtracking

Fix — First-Fail (`ff`) heuristic:

- At each step, pick variable with **smallest remaining domain**
- Surfaces failures earlier, cuts branches sooner

One-line fix

```
labeling([ff,
  minimize(Cost)],
  AllVars)
```

Other candidates: `ffc`, `bisect`, `min`/`max` value ordering.

Factor 2 — Weak Lower Bounds

SICStus branch-and-bound

- Finds a feasible solution, records cost as upper bound
- Continues search for strictly cheaper solutions
- Lower bounds from **constraint propagation only**—often loose
- Cannot prove a subtree is suboptimal without exploring it

OR-Tools CP-SAT

- Runs an **LP relaxation** at each node
- LP gives **tight fractional lower bounds**
- Can prune entire subtrees with a *proof* they cannot improve
- Dramatically reduces the effective search space

Factor 3 — Propagation vs Learning

CLP(FD) — Arc Consistency

- Domain shrinks ⇒ re-check connected constraints
- Transitively prunes domains (AC-3 / AC-4
- **Sound but shallow**: cannot reason about combinations of decisions
- **Backtracks and forgets**

CP-SAT — CDCL

- Encodes everything as Boolean clauses
- On contradiction, **analyses root cause**
- Derives a **no-good clause** and adds it permanently
- Clause prunes regions in *future* branches
- **Learns and remembers**

CLP(FD) backtracks; CP-SAT prunes with proof.

Factor 4 — Unhandled Symmetry

P1 and P2 have **identical coverage radii** ($r = 1$).

Consequence

Any solution with P1 and P2 antenna placements swapped has **exactly the same cost**. Without symmetry breaking, the solver explores **both symmetric variants**—effectively doubling work across a large portion of the search tree.

Fix — Lexicographic ordering constraint:

```
lex_chain([P1, P2])
```

Forces a canonical ordering, eliminating the symmetric half of the search space. OR-Tools handles this **internally**; the SICStus model does not.

OR-Tools Timing Context

Important: OR-Tools overhead

The 330–500 ms OR-Tools figures are dominated by:

1. Python interpreter startup
2. JSON map loading and parsing
3. Model construction and coverage precomputation

The actual **CP-SAT search time is likely single-digit milliseconds** for these grid sizes.

SICStus at 4×4 (≈380 ms) is **comparable** once Prolog startup and library loading are accounted for.

The divergence becomes visible at **5×5** where CLP(FD) search time dominates (8–14 s), and the solver cannot complete **6×6** within 60 s.

Outline

Summary

What we built

- Full CSP/CSOP formulation of a realistic telecom network placement problem
- Two complete, equivalent implementations in OR-Tools and SICStus
- Shared map generator and benchmark harness
- Quantified comparative study across 5 grid size

Key findings

- **Model equivalence** confirmed on all shared instances
- OR-Tools scales to $8\times8+$ with ease; SICStus times out at 6×6
- The gap is **not a fundamental CLP(FD) limitation**—it is solver configuration
- CDCL + LP bounds vs pure propagation is the decisive factor

Course Connections

This project applies the core PLR/CLP competencies:

- **Declarative modelling**: problem expressed as constraints, not procedural search
- **CLP(FD) in SICStus Prolog**: finite domain variables, arc consistency, branch-and-bound
- **Constraint propagation**: how domain reductions propagate through the constraint graph
- **Search strategies**: impact of variable ordering and value selection heuristics
- **Comparative study**: same model, different solvers—understanding *why* performance differs

The performance gap between solvers is itself a concrete demonstration of course theory: why heuristics, symmetry breaking, and learned clauses matter in practice.

Future Work

1. **SICStus search heuristics**: add `ff` variable ordering; expected to make 6×6 solvable
2. **Symmetry breaking**: `lex_chain([P1, P2])` to eliminate symmetric search space
3. **Extended OR-Tools benchmarks**: test 10×10, 12×12, 15×15 to find CP-SAT's own limits
4. **Mathematical formalization**: complete ILP/CSP notation for report submission
5. **Grid visualization**: matplotlib overlay of terrain, towers, and coverage radii

Questions?

Multi-Tenant Mobile Antenna Placement Optimization

Martim Pinheiro (up20509641) · Adam Oprchal (up202512712)

Thank you!